

(a)

```
In [12]: import numpy as np
def f(x):
    return x**2 - np.exp(-x) - 2 * np.sin(x)

def deriv_f(x):
    return 2*x + np.exp(-x) - 2 * np.cos(x)

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    x = initial_guess
    for i in range(max_iteration):
        f_x = f(x)
        deriv_x = deriv_f(x)

        if deriv_x == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        x_new = x - f_x / deriv_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new

    Print("Maximum iteration reached. Method may not have converged.")
    return x

Root = new_raphson(f , deriv_f, 1.5)

if Root is not None:
    print("The root found is: ",Root)
```

The root found is: 1.4895886408077366

(b)

```
In [18]: import numpy as np
def f(t):
    return 10 * t - 4.9 * t**2

def f_prime(t):
    return 10 - 9.8*t

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    t = initial_guess
    for i in range(max_iteration):
        f_t = f(t)
        deriv_t = deriv_f(t)
```

```

        if deriv_t == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        t_new = t - f_t / deriv_t

        if abs(t_new - t) < tolerance:
            return t_new

        t = t_new

    Print("Maximum iteration reached. Method may not have converged.")
    return t

Root = new_raphson(f , f_prime, initial_guess)

if Root is not None:
    print("Time take projectile returns to initial height: ",Root,"Seconds")

```

Time take projectile returns to initial height: 2.040816326530625 Seconds

(c)

```

In [13]: import numpy as np
def f(x):
    return x**2 - 3

def f_prime(x):
    return 2*x

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    x = initial_guess
    for i in range(max_iteration):
        f_x = f(x)
        deriv_x = deriv_f(x)

        if deriv_x == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        x_new = x - f_x / deriv_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new

    Print("Maximum iteration reached. Method may not have converged.")
    return x

Root = new_raphson(f , f_prime, 1.5)

if Root is not None:
    print("The Square root of 3 is: ",Root)

```

The Square root of 3 is: 1.7320508075688772

(d)

```
In [16]: import numpy as np
def f(L):
    return L - (100*9.8)/(4*np.pi*np.pi)

def deriv_f(L):
    return 1

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    L = initial_guess
    for i in range(max_iteration):
        f_L = f(L)
        deriv_L = deriv_f(L)

        if deriv_L == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        L_new = L - f_L / deriv_L

        if abs(L_new - L) < tolerance:
            return L_new

        L = L_new

    Print("Maximum iteration reached. Method may not have converged.")
    return L

Root = new_raphson(f , deriv_f, 20)

if Root is not None:
    print("Length of the pendulum for T=10 :", Root, "meters")
```

Length of the pendulum for T=10 : 24.82368999237276 meters

(e)

```
In [19]: import numpy as np
def f(t):
    return 5 * (1 - np.exp(-t / 10)) - 2.5

def f_prime(t):
    return np.exp(-t / 10)/2

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    t = initial_guess
    for i in range(max_iteration):
        f_t = f(t)
        deriv_t = deriv_f(t)
```

```

    if deriv_t == 0:
        print("Derivative is zero. Newton-Raphson method cannot proceed.")
        return None

    t_new = t - f_t / deriv_t

    if abs(t_new - t) < tolerance:
        return t_new

    t = t_new

    Print("Maximum iteration reached. Method may not have converged.")
    return t

Root = new_raphson(f , f_prime, 10)

if Root is not None:
    print("Voltage across the capacitor :",Root,"V")

```

Voltage across the capacitor : 6.931471805599453 V